从瞬时速度到一步生成

MeanFlow 与 Improved Mean Flows

从基本公式一步步推导到训练算法

JVP、Stop Gradient、u-loss / v-loss 与整体速度场

参照 flow_matching_latex 的 B5 技术书风格排版

数学记号对齐 arXiv:2505.13447v1 和 arXiv:2512.02012v2

阅读说明

这份笔记衔接已有的 *Flow Matching* 学习材料，严格只安排三章：第 1 章解释原始 MeanFlow 的数学原理；第 2 章解释 Improved Mean Flows（以下记为 iMF）的数学原理与真正的改进动机；第 3 章用表格比较普通 Flow Matching、MeanFlow 和 iMF。

写法特意针对学习中容易卡住的问题：**为什么每条训练插值线是直线，生成时仍需多步？为什么积分求导会得到速度？为什么全导数包含 Jacobian？JVP 到底是数学定义还是 PyTorch 函数？Stop Gradient 为什么需要？iMF 究竟是换了 Loss，还是换了 JVP 方向？训练时是否需要两个网络？**这些问题均放在第一次涉及的位置回答，计算步骤尽量不省略。

记号与来源。 特别注意时间方向：参考文件 `flow_matching_latex` 使用 $t = 0$ **是噪声端**、$t = 1$ **是数据端**，但原始 MeanFlow 与 iMF 的 arXiv 原文正好相反。为了让 JVP 目标、原文 Algorithm 1 与 One-Step 推理公式完全一致，**本书的所有数学推导统一沿用 MeanFlow 原文的方向**：

$$z_t = (1 - t)x + t\epsilon, \qquad t = 0 \text{ 是数据端}, \quad t = 1 \text{ 是噪声端}.$$

这只是时间参数化方向的选择，不是三种方法的本质差别。**不要把参考笔记中的** x_0、x_1 **端点记法直接代入本书的加减号**；改变时间方向时，速度符号和平均速度的锚点定义需要一起调整。原始 MeanFlow 的公式对照 arXiv:2505.13447v1；iMF 的公式、表格与所引用原图，核对用户上传的 arXiv:2512.02012v2 LaTeX 源码（主要是 `sections/background.tex`、`sections/method.tex`、`sections/appendix.tex`）。**笔记中的示范数字是教学例子，并非论文实验数据**。文中仅引用 iMF 作者提供的原图文件，不自行绘制论文图片。公式均不使用外框。

三章学习路线。

1. 先分清**采样路径速度**与**整体速度场**，再看区间平均速度与 MeanFlow Identity。
2. 再分清**等价的损失改写**与**真正改变算法的 JVP 方向**，最后理解 CFG 和辅助速度头。
3. 最后用三方法对照表检查，自己能否从训练数据推到推理公式。

目录

阅读说明 **ii**

第一章 MeanFlow 数学原理 **1**

1.1 先从 Flow Matching 的瞬时速度开始 1
1.2 区间平均速度到底是什么 3
1.3 逐行推导 MeanFlow Identity 4
1.4 全导数、Jacobian 和 JVP 为什么同时出现 5
1.5 MeanFlow 如何把恒等式变成训练 Loss 7
1.6 从一次训练更新到一次推理 8
1.7 原始 MeanFlow 常见疑惑的集中回答 9

第二章 Improved Mean Flows 的数学原理 **11**

2.1 先明确：iMF 不推翻 MeanFlow Identity 11
2.2 第一步：把 u-loss 精确改写为 v-loss 11
2.3 第二步：为什么 JVP 方向要从样本速度换成整体速度 13
2.4 两个预测量意味着训练两个大模型吗? 15
2.5 Stop Gradient 在 iMF 中被删除了吗? 16
2.6 Flexible CFG：为什么另有第二项改进 17
2.7 In-context Conditioning：模型怎样接收这么多条件 18
2.8 论文怎样验证改进? 18
2.9 iMF 训练伪代码与最后的疑问 20

第三章 Flow Matching、MeanFlow 和 iMF 对比 **21**

3.1 核心数学对象与训练机制总表 21
3.2 最关键的三组损失，逐式对照 22
3.3 训练与推理各需要什么：快速检查表 23
3.4 你应该能回答的七个自检问题 24
3.5 结论：真正的技术演进 24
论文与源码出处 25

第 1 章

MeanFlow 数学原理

1.1 先从 Flow Matching 的瞬时速度开始

要解决的生成问题。 给定数据样本 $x \sim p_{\text{data}}$ 与高斯噪声 $\epsilon \sim p_{\text{prior}}$，我们希望从噪声生成数据。在本书的时间约定里，真实数据在 $t = 0$，噪声在 $t = 1$。训练时随机挑选一对 x, ϵ，构造线性插值：

$$z_t = (1 - t)x + t\epsilon, \qquad 0 \le t \le 1. \tag{1.1}$$

分别代入端点：

$$z_0 = x, \qquad z_1 = \epsilon.$$

求导时把已经抽出的 x, ϵ 当作常量：

$$\frac{\mathrm{d}z_t}{\mathrm{d}t} = \frac{\mathrm{d}}{\mathrm{d}t}\big[(1 - t)x + t\epsilon\big] \tag{1.2}$$

$$= -x + \epsilon = \epsilon - x. \tag{1.3}$$

把这条**具体插值路径**的速度记为

$$v_{\mathrm{c}} = \epsilon - x. \tag{1.4}$$

这又叫 conditional velocity（条件速度、采样路径速度）。这里的「条件」不是文字提示词，而是指这条训练路径由具体的 x, ϵ 决定。

为什么每条线是直线，生成轨迹却可能弯曲？ 模型预测时并不知道创建 z_t 的具体 x 和 ϵ。同一个 z_t 可以由不同的样本对产生。例如 $t = 0.5$ 时：

样本	x	ϵ	$z_{0.5}$	v_{c}
A	0	2	1	2
B	−1	3	1	4

两个样本经过同一个位置 $z_{0.5} = 1$，却有不同的条件速度 2 和 4。如果两种情形条件概率各为 $1/2$，则模型真正需要预测的**整体速度场（marginal velocity field）**在这个位置为 3：

$$v(z_t, t) = \mathbb{E}\big[v_{\mathrm{c}} \mid z_t, t\big]. \tag{1.5}$$

在真实训练分布中，当然不只两种样本；上述数字仅用于说明「同一个 z_t，可能对应多个 v_{c}」。

为什么可以用 v_{c} 训练 v_θ？ 普通 Flow Matching 对单条样本使用平方损失：

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}\big\|v_\theta(z_t, t) - v_{\mathrm{c}}\big\|_2^2. \tag{1.6}$$

固定 z_t, t 后，把样本速度写成 $v_{\mathrm{c}} = v(z_t, t) + \eta$，其中 $\mathbb{E}[\eta \mid z_t, t] = 0$。于是

$$\mathbb{E}\big[\|v_\theta - v_{\mathrm{c}}\|^2 \mid z_t, t\big] = \mathbb{E}\big[\|(v_\theta - v) - \eta\|^2 \mid z_t, t\big] \tag{1.7}$$

$$= \|v_\theta - v\|^2 + \mathbb{E}\big[\|\eta\|^2 \mid z_t, t\big]. \tag{1.8}$$

中间的交叉项为零，因为 η 的条件期望是零；第二项不由 v_θ 决定。因此平方误差回归倾向于学习条件期望 v，**而不是逐一记住每个** $\epsilon - x$。

为何多步求解？ 普通 Flow Matching 训练瞬时速度 v_θ，推理时通过 ODE 沿整个速度场运动：

$$\frac{\mathrm{d}z_t}{\mathrm{d}t} = v_\theta(z_t, t), \qquad z_1 = \epsilon. \tag{1.9}$$

从 1 向 0 积分：

$$z_0 = z_1 - \int_0^1 v_\theta(z_\tau, \tau)\ \mathrm{d}\tau. \tag{1.10}$$

当前瞬时速度通常不能表示整段位移，所以数值求解器要在多个时间点重复调用网络。即使(1.1)对每对训练样本是一条直线，边缘速度场一般仍可使生成路径弯曲。这并非仅由网络预测误差造成。

本节要点： 训练样本提供 $v_{\mathrm{c}} = \epsilon - x$；生成时网络要学习的是 $v(z_t, t) = \mathbb{E}[v_{\mathrm{c}} \mid z_t, t]$。这是 MeanFlow 和 iMF 之后一切讨论的基础。

原文对应：原始 MeanFlow 论文 Sec. 3；iMF 源码 `sections/background.tex`。

1.2 区间平均速度到底是什么

从总位移的定义出发。 沿**整体速度场** $v(z_t, t)$ 生成一条满足 ODE 的轨迹。固定时间区间起点 r，令 $0 \le r < t \le 1$。从 r 到 t 的总位移满足

$$z_t - z_r = \int_r^t v(z_\tau, \tau)\ \mathrm{d}\tau. \tag{1.11}$$

平均速度等于总位移除以区间长度，所以定义

$$u(z_t, r, t) \triangleq \frac{1}{t-r}\int_r^t v(z_\tau, \tau)\ \mathrm{d}\tau = \frac{z_t - z_r}{t-r}. \tag{1.12}$$

为什么写成 $u(z_t, r, t)$，而不是只写 $u(t)$？ 因为不同轨迹即使时间相同，其当前位置 z_t 也不同；在固定整体速度场并假设 ODE 解良好的情况下，当前端点 z_t 和区间 $[r, t]$ 决定该段轨迹，进而决定平均速度。这里的 z_t 是生成空间中的向量，不必是现实世界中的空间坐标。

平均速度不是起点速度，也不是终点速度。 假设一维速度 $v(\tau) = 2\tau$，在 $[0, 1]$ 内：

$$v(0) = 0, \qquad v(1) = 2, \qquad u = \int_0^1 2\tau\ \mathrm{d}\tau = 1.$$

用终点瞬时速度乘以时间，粗略得到位移 2；用真实平均速度乘以时间，位移恰好是 1。这说明 MeanFlow 学到平均速度后，为什么**有可能一次跨越整个区间**。

直接从平均速度得到一步生成。 把(1.12)移项：

$$(t-r)u(z_t, r, t) = z_t - z_r, \tag{1.13}$$

$$z_r = z_t - (t-r)u(z_t, r, t). \tag{1.14}$$

选取完整区间 $(r, t) = (0, 1)$：

$$x = z_0 = \epsilon - u(\epsilon, 0, 1). \tag{1.15}$$

如果网络 u_θ 准确预测这个平均速度，则只要一次网络调用：

$$x_{\text{pred}} = \epsilon - u_\theta(\epsilon, 0, 1). \tag{1.16}$$

NFE=1 指推理阶段只求值一次 u_θ；不代表一次训练、一个图像像素，或机器人只执行一个动作。上式成立的前提是 u_θ 学到了足够准确的区间平均速度。

区间缩小会发生什么？ 平均速度在 $r \to t$ 时退化为瞬时速度：

$$\lim_{r \to t} u(z_t, r, t) = v(z_t, t). \tag{1.17}$$

所以 u 既能表示短区间速度，也能表示完整区间速度。这个边界关系稍后还用于 iMF 的 v_θ 估计。

本节要点： 普通 Flow Matching 预测「此刻怎么走」；MeanFlow 预测「整个区间平均怎么走」。但真正的平均速度不能直接从一条训练直线速度 v_c 推出，因此还需要训练恒等式。

1.3 逐行推导 MeanFlow Identity

第一步：把平均速度定义两边乘以 $t-r$。

$$(t-r)u(z_t, r, t) = \int_r^t v(z_\tau, \tau)\ \mathrm{d}\tau. \tag{1.18}$$

在下面求导时，**固定 r，只让终点 t 改变**。

第二步：左边使用乘积求导法则。 令 $f(t) = t - r$，$g(t) = u(z_t, r, t)$，则

$$\frac{\mathrm{d}}{\mathrm{d}t}\big[(t-r)u\big] = \frac{\mathrm{d}(t-r)}{\mathrm{d}t}\,u + (t-r)\frac{\mathrm{d}u}{\mathrm{d}t} \tag{1.19}$$

$$= u + (t-r)\frac{\mathrm{d}u}{\mathrm{d}t}. \tag{1.20}$$

注意 u 的输入 z_t 也随 t 变，所以这里必须写**全导数** $\mathrm{d}u/\mathrm{d}t$，不能偷换成只对显式时间求偏导数。

第三步：右边使用微积分基本定理。

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_r^t v(z_\tau, \tau)\ \mathrm{d}\tau = v(z_t, t). \tag{1.21}$$

为什么？ 把终点从 t 增加到 $t+\Delta t$，新增加的积分大约为 $v(z_t,t)\Delta t$；除以 Δt 再令 $\Delta t \to 0$，得到端点的瞬时速度。这里积分变量 τ 是哑变量，不是需要再次求导的自变量。

第四步：令两边导数相等并移项。

$$u + (t-r)\frac{\mathrm{d}u}{\mathrm{d}t} = v, \tag{1.22}$$

$$u = v - (t-r)\frac{\mathrm{d}u}{\mathrm{d}t}. \tag{1.23}$$

这就是原论文的 **MeanFlow Identity**。直觉是：当前瞬时速度 v 与区间平均速度 u 并不相同，差别通过平均速度随轨迹变化的导数来校正。

为什么这比直接计算积分有用？ 定义式需要知道整个区间 $[r,t]$ 上的速度积分，而恒等式只涉及当前位置的速度和导数。训练时虽然不知道真实 u，却可以用网络近似这个导数，从而构造局部、可计算的监督目标。

1.4 全导数、Jacobian 和 JVP 为什么同时出现

第一层：区分偏导数与全导数。 网络输入有三个：

$$u_\theta = u_\theta(z_t, r, t).$$

随时间变化时，z_t 变化，t 也变化，而 r 固定。对一个标量输出，链式法则为

$$\frac{\mathrm{d}u_\theta}{\mathrm{d}t} = \frac{\partial u_\theta}{\partial z_t}\frac{\mathrm{d}z_t}{\mathrm{d}t} + \frac{\partial u_\theta}{\partial r}\frac{\mathrm{d}r}{\mathrm{d}t} + \frac{\partial u_\theta}{\partial t}\frac{\mathrm{d}t}{\mathrm{d}t}. \tag{1.24}$$

其中

$$\frac{\mathrm{d}z_t}{\mathrm{d}t} = v(z_t,t), \qquad \frac{\mathrm{d}r}{\mathrm{d}t} = 0, \qquad \frac{\mathrm{d}t}{\mathrm{d}t} = 1.$$

如果网络输入、输出都是高维向量，就需要把空间偏导写成 Jacobian：

$$\frac{\mathrm{d}u_\theta}{\mathrm{d}t} = J_z u_\theta \, v(z_t,t) + \partial_t u_\theta. \tag{1.25}$$

这里 $J_z u_\theta \in \mathbb{R}^{d\times d}$，$v \in \mathbb{R}^d$；两者相乘得到 d 维向量。第 i 个分量就是 $\sum_{j=1}^{d}(\partial u_i/\partial z_j)v_j$。

第二层：JVP 的数学定义。 对可微函数 $f : \mathbb{R}^n \to \mathbb{R}^m$，在输入 q 沿方向 w 的 Jacobian-Vector Product 定义为

$$\mathrm{JVP}(f; q; w) = J_f(q)w = \left.\frac{\mathrm{d}}{\mathrm{d}s} f(q + sw)\right|_{s=0}. \tag{1.26}$$

JVP 首先是一种数学运算。`torch.func.jvp` 和 `jax.jvp` 只是实现这种运算的函数接口。并不是 MeanFlow 自己发明了 JVP。

第三层：MeanFlow 的 JVP 方向为什么是 $(v, 0, 1)$？ 把三个输入合并为 $q = (z_t, r, t)$。沿时间的变化方向为

$$\frac{\mathrm{d}q}{\mathrm{d}t} = \left(\frac{\mathrm{d}z_t}{\mathrm{d}t}, \frac{\mathrm{d}r}{\mathrm{d}t}, \frac{\mathrm{d}t}{\mathrm{d}t}\right) = (v, 0, 1). \tag{1.27}$$

因此

$$\mathrm{JVP}\left(u_\theta; (z_t, r, t); (v, 0, 1)\right) = J_z u_\theta\, v + \partial_r u_\theta \cdot 0 + \partial_t u_\theta \cdot 1 \tag{1.28}$$

$$= \frac{\mathrm{d}u_\theta}{\mathrm{d}t}. \tag{1.29}$$

括号里的 $(v, 0, 1)$ 不是超参数，而是三个输入各自关于时间的变化率。

一个完整的 JVP 数字例子。 取 $f(q_1, q_2) = (q_1^2 + q_2,\ 3q_1 + q_2^2)^\mathsf{T}$。其 Jacobian 为

$$J_f(q) = \begin{bmatrix} 2q_1 & 1 \\ 3 & 2q_2 \end{bmatrix}.$$

在 $q = (1, 2)^\mathsf{T}$、$w = (1, 2)^\mathsf{T}$ 处，

$$J_f(q)w = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 11 \end{bmatrix}.$$

这说明 JVP 返回的是**函数沿指定方向的瞬时变化率**，并不是把输入真的移动一步之后的输出。

在 PyTorch 中如何写？

```python
from torch.func import jvp

# net(z, r, t): predict average velocity u
```

```
u_pred, du_dt = jvp(
    net,
    (z_t, r, t),
    (v_sample, torch.zeros_like(r),
     torch.ones_like(t))
)
```

JVP 会同时返回函数正常输出 u_θ 和沿指定方向的导数。对于高维动作或图像，不需要先显式构造庞大的完整 Jacobian 矩阵。

易错点： 这里训练代码使用 v_{sample} 作为 JVP 方向是原始 MeanFlow 的实现近似；理论式(1.25)本来使用整体速度场 $v(z_t, t)$。这正是第 2 章 iMF 要改进的地方。

1.5 MeanFlow 如何把恒等式变成训练 Loss

真实的平均速度 u 是未知的，因此我们用神经网络 u_θ 近似。根据 MeanFlow Identity：

$$u_{\text{tgt}} = v_{\text{c}} - (t-r)\,\text{JVP}\left(u_\theta; (z_t, r, t); (v_{\text{c}}, 0, 1)\right), \qquad v_{\text{c}} = \epsilon - x. \tag{1.30}$$

这里的 u_{tgt} 读作 ***u target***（平均速度训练目标）。核心损失为

$$\mathcal{L}_{\text{MF}} = \mathbb{E}_{x,\epsilon,r,t}\left\|u_\theta(z_t, r, t) - \text{sg}\big(u_{\text{tgt}}\big)\right\|_2^2. \tag{1.31}$$

训练目标为什么也包含当前网络？ 因为真实的 $\mathrm{d}u/\mathrm{d}t$ 无法得到，MeanFlow 用当前网络的 $\mathrm{d}u_\theta/\mathrm{d}t$ 来近似它，构成一种**自举（bootstrap）训练**：当前网络产生参考目标，本轮向该目标回归，下一轮用更新后的网络重新计算。它并不是有一位教师提供完全固定的真实平均速度标签。

Stop Gradient 到底停止了什么？ sg 在前向计算时不改变数值，但在反向传播时把其内部当作常量。例如在 PyTorch 中相当于 `u_target.detach()`。**它只停止目标分支的参数梯度，不停止预测分支的更新，也不阻止下次重新计算目标。**

为了看出差异，令 $\Delta = t - r$、$g_\theta = \mathrm{d}u_\theta/\mathrm{d}t$、$e_\theta = u_\theta - v_{\text{c}} + \Delta g_\theta$。若完整地最小化平方残差，得到

$$\nabla_\theta \|e_\theta\|^2 = 2\left(\partial_\theta u_\theta + \Delta\,\partial_\theta g_\theta\right)^{\mathsf{T}} e_\theta. \tag{1.32}$$

其中 g_θ 本来就是网络关于输入的导数；再求 $\partial_\theta g_\theta$ 会出现 $\partial^2 u_\theta/(\partial\theta\,\partial z)$ 等混合高阶导数。采用 Stop Gradient 后，本轮更新方向成为

$$\widetilde{\nabla}_\theta \mathcal{L}_{\text{MF}} = 2(\partial_\theta u_\theta)^{\mathsf{T}} e_\theta. \tag{1.33}$$

它称为**半梯度（semi-gradient）**，并不表示把梯度大小除以二，而是仅对预测分支求导。其优点主要是避免在参数更新时再反向通过 JVP；但**半梯度不保证在所有问题上比完整梯度更稳定、更容易收敛**。

一个零基础数值例子。 假设本轮 $v_{\text{c}} = 6$，$r = 0.2$，$t = 0.6$；网络预测 $u_\theta = 5.2$，JVP 结果为 1.5。则

$$u_{\text{tgt}} = 6 - 0.4 \times 1.5 = 5.4, \qquad \ell = (5.2 - 5.4)^2 = 0.04.$$

注意这里只是展示**目标怎么计算**，不是断言真实区间平均速度必定是 5.4。

为什么当 $r = t$ 时变回普通 Flow Matching？ 此时 $t - r = 0$，所以 $u_{\text{tgt}} = v_{\text{c}}$；损失化为

$$\|u_\theta(z_t, t, t) - (\epsilon - x)\|^2 .$$

这正是标准速度回归的形式；训练样本含有 $r = t$ 的情况可以直接锚定网络的瞬时速度预测。论文训练时还探讨时间采样和自适应损失权重，这些是实际超参数设计，不改变本节核心恒等式。

原文对应：原始 MeanFlow 论文 Sec. 4.1（公式 (6)–(11)，Algorithm 1）、Sec. 4.3；iMF 源码 `sections/background.tex`。

1.6 从一次训练更新到一次推理

训练的概念伪代码。

```
# x: batch of real data
# fn(z, r, t): model predicting u
# Note: conceptual pseudocode, not a full runnable program.
r, t = sample_interval()  # 0 <= r <= t <= 1
eps = randn_like(x)
z_t = (1 - t) * x + t * eps
v_sample = eps - x
```

```
u_pred, du_dt = jvp(
    fn, (z_t, r, t), (v_sample, 0, 1)
)
u_target = v_sample - (t - r) * du_dt
loss = mse(u_pred, stopgrad(u_target))
update_model(loss)
```

这里 jvp 的第二组输入表示**方向**，而不是另一组需要喂给网络的训练样本；stopgrad 只在损失反向传播时起作用。

推理的概念伪代码。

```
eps = randn(shape)
x_pred = fn(eps, r=0, t=1)
x_pred = eps - x_pred
```

训练要计算 JVP，推理却不需要 JVP：因为推理只取 $u_\theta(\epsilon, 0, 1)$，不构造训练目标，也不反向传播。因而 MeanFlow 的核心代价交换是**训练增加导数计算，换取推理减少积分步骤**。

沿着区间分段也可以采样。 若不选择整段 $[0, 1]$，而是依次选择若干子区间，仍可使用更新式(1.14)。因此**能进行 One-Step 与所有情况下 One-Step 都最好**不是同一个结论。单步结果受平均速度预测误差影响；若需要较高精度，也可以使用更多次网络调用。

1.7 原始 MeanFlow 常见疑惑的集中回答

问：u 和 v 是两个网络吗？ 原始 MeanFlow 训练的是**一个平均速度网络** u_θ。公式里的 v 是理论上的整体瞬时速度场，v_c 是从训练数据直接计算的监督信号；原始训练算法不需要再训练一个独立的 v_θ 网络。

问：JVP 是否就是 `torch.func.jvp`？ 前者是数学概念，后者是 PyTorch 的实现。数学定义为 $J_f(q)w$，PyTorch 接口负责高效算出这个乘积而不必存储整个 Jacobian。

问：为什么 JVP 的方向不是 $(v_{\mathrm{c}}, 1, 1)$？ 区间起点 r 在导出 MeanFlow Identity 时是固定的，$\mathrm{d}r/\,\mathrm{d}t = 0$；时间终点对自己的变化率是 1，因此必须是 $(v_{\mathrm{c}}, 0, 1)$。

问：为什么既有 v 又有 v_{c}？ $v(z_t, t)$ 是固定当前位置后的**条件期望**；v_{c} 是某条抽样直线的速度。后者容易获得但带有采样随机性。使用 v_{c} 监督 Flow Matching 是合理的，但把它放入 MeanFlow 的 JVP 方向会产生额外问题，下一章专门研究。

问：既然每条训练路径速度恒定，为什么 $\mathrm{d}u_\theta/\,\mathrm{d}t$ 不总是零？ 恒定的是**单条抽样路径**的 $\epsilon - x$；真实生成流依赖整体速度场，沿整体轨迹的平均速度不必恒定。网络 u_θ 试图拟合这个整体的平均速度，不是逐条记忆抽样直线的常量。

问：原始 MeanFlow 论文是否说 Stop Gradient 一定优于完整梯度？ 没有这个普遍性保证。作者强调它可避免双重反向传播（double backpropagation）带来的高阶求导成本，并在自己的配置下形成可行的自举更新。

第 2 章

Improved Mean Flows 的数学原理

2.1 先明确：iMF 不推翻 MeanFlow Identity

iMF 的出发点仍然是

$$u = v - (t-r)\frac{\mathrm{d}u}{\mathrm{d}t}. \tag{2.1}$$

它没有改变 u 是**沿整体速度场的区间平均速度**这个定义，也没有把单步生成改成多步生成。

作者研究了两个主要困难：

1. **训练目标：**原始 MF 的 u_{tgt} 包含网络自身的 JVP；若把原损失解释为瞬时速度回归，会发现这个预测函数额外使用了**样本标签** v_{c} 作为输入。
2. **CFG：**原始 MF 的引导尺度一般在训练阶段固定，推理时不易自由修改；iMF 将引导尺度及区间变成网络条件，并改进条件编码方式。

本章先完整讲第一个动机，再讲相应的网络实现与 CFG。**最重要的区分是：移项写成 v-loss 只是等价重述；改变 JVP 的速度方向，才改变实际学习过程**。

2.2 第一步：把 u-loss 精确改写为 v-loss

从恒等式移项，绝不是提出新定理。

$$u = v - (t-r)\frac{\mathrm{d}u}{\mathrm{d}t} \quad \Longleftrightarrow \quad v = u + (t-r)\frac{\mathrm{d}u}{\mathrm{d}t}.$$

原始 MF 采用**预测 u，构造 u 的目标**：

$$g_\theta^{\text{old}} \triangleq \text{JVP}\left(u_\theta; (z_t, r, t); (v_{\text{c}}, 0, 1)\right), \tag{2.2}$$

$$u_{\text{tgt}} = v_{\text{c}} - (t-r)g_\theta^{\text{old}}, \tag{2.3}$$

$$\mathcal{L}_u = \|u_\theta - \text{sg}\big(u_{\text{tgt}}\big)\|^2. \tag{2.4}$$

设 $\Delta = t - r$。由于 Stop Gradient 在**前向计算中不改变值**，并且 v_c 和 Δ 对 θ 都是常量，我们可以重新组织：

$$V_\theta^{\text{old}} \triangleq u_\theta + \Delta\ \text{sg}\Big(g_\theta^{\text{old}}\Big)\,. \tag{2.5}$$

新写法为

$$\mathcal{L}_v^{\text{old}} = \|V_\theta^{\text{old}} - v_c\|^2. \tag{2.6}$$

为什么两种写法的数值完全相同？ 旧式残差为

$$u_\theta - u_{\text{tgt}} = u_\theta - [v_c - \Delta g_\theta^{\text{old}}] = u_\theta + \Delta g_\theta^{\text{old}} - v_c.$$

新式残差为

$$V_\theta^{\text{old}} - v_c = u_\theta + \Delta\ \text{sg}\Big(g_\theta^{\text{old}}\Big) - v_c.$$

两者的**前向数值相同**。反向传播时，旧式 $\text{sg}\big(u_{\text{tgt}}\big)$ 与新式 $\text{sg}\big(g_\theta^{\text{old}}\big)$ 都会阻止梯度经过 g_θ^{old}，因此两者的**参数更新方向也相同**。

结论：在仍使用同一个 v_c 方向且 Stop Gradient 不变时，从 u-loss 到 v-loss 并没有算法收益。

重写公式的真正价值：暴露了隐藏输入。 根据链式法则，

$$V_\theta^{\text{old}} = u_\theta + \Delta\ \text{sg}(J_z u_\theta\, v_c + \partial_t u_\theta)\,. \tag{2.7}$$

这说明 V_θ^{old} 除了依赖 (z_t, r, t)，**还需要知道这次训练样本的** $v_c = \epsilon - x$。于是把它叫作「标准地仅由当前位置决定的瞬时速度预测」并不恰当。iMF 的关键问题不是旧方法无效，而是**它的复合预测函数额外接收了一个随机标签相关的方向输入**。

原文对应：iMF `sections/method.tex`，小节 “MeanFlow as v-loss”；原文 Eq. (v-loss) 相关公式。

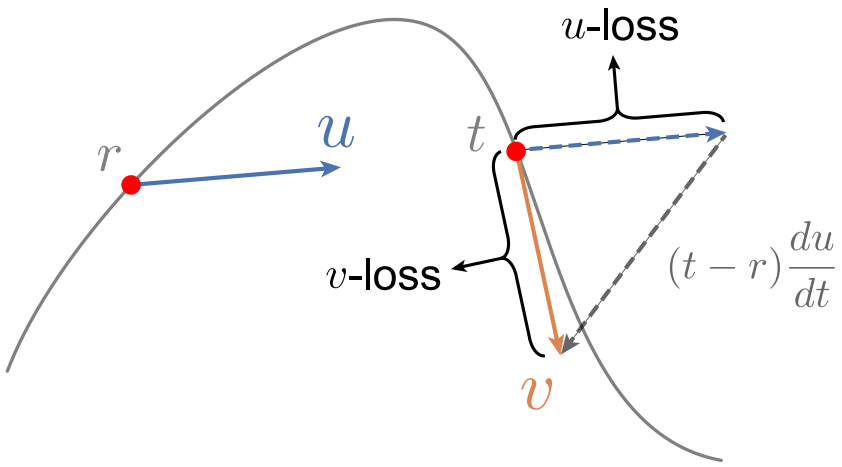


图 2.1: iMF 原论文关于「MeanFlow 可重参数化为 v-loss」的原图。图片直接取自用户上传的 arXiv:2512.02012v2 源码 `figs/demo.pdf`，未重绘。

2.3 第二步：为什么 JVP 方向要从样本速度换成整体速度

先找出理论中真正需要的方向。 MeanFlow Identity 的导数是沿着**整体速度场**的真实轨迹计算：

$$\frac{\mathrm{d}u}{\mathrm{d}t} = J_z u \, v(z_t, t) + \partial_t u. \tag{2.8}$$

因此 JVP 的理论方向是 $\big(v(z_t,t), 0, 1\big)$，不是单条训练样本的 $\big(\epsilon - x, 0, 1\big)$。

原始 MF 做了什么近似？ 整体速度场直接求不到，于是原始 MF 使用 $v_{\mathrm{c}} = \epsilon - x$ 替代。固定 z_t, t 后，可以写成

$$v_{\mathrm{c}} = v(z_t, t) + \eta, \qquad \mathbb{E}[\eta \mid z_t, t] = 0. \tag{2.9}$$

将其代入 JVP：

$$\mathrm{JVP}(u_\theta; v_{\mathrm{c}}) = J_z u_\theta \, v_{\mathrm{c}} + \partial_t u_\theta \tag{2.10}$$

$$= J_z u_\theta \, v + \partial_t u_\theta + J_z u_\theta \, \eta. \tag{2.11}$$

末项 $J_z u_\theta \eta$ 是来自抽样路径的随机方向偏差。它的条件均值是零，但**条件方差未必小**。如果 $J_z u_\theta$ 对某些方向放大很强，随机波动也会被放大。

为什么原始做法不是「数学错误」？ 对于固定的 (z_t, r, t) 和当前网络参数，由 JVP 对输入方向的**线性性**，有

$$\mathbb{E}\big[\mathrm{JVP}(u_\theta; v_c) \mid z_t, t\big] = \mathrm{JVP}\left(u_\theta; v(z_t, t)\right). \tag{2.12}$$

所以旧 JVP 方向**在条件期望意义上并非必然有偏**；问题主要是方差、训练目标依赖采样标签以及训练过程的数值行为，而不是 MeanFlow Identity 被推翻。

一个能够算出方差的简单例子。 仍然取两个等概率条件速度 2 和 4，故 $v = 3$。设一维 $J_z u_\theta = 6$，区间长度 $\Delta = 0.4$，忽略此例中的显式时间偏导。旧式复合预测残差中随机部分为

$$V_\theta^{\mathrm{old}} - v_c = \left(u_\theta + \Delta \cdot 6 v_c\right) - v_c = u_\theta + (2.4 - 1) v_c.$$

因此随机速度的系数是 1.4。若 $\mathrm{Var}(v_c \mid z_t, t) = 1$，则残差的条件方差为 $1.4^2 = 1.96$。

而改进后 $v_\theta(z_t, t)$ 对固定的 (z_t, t) 是一个确定值；复合预测不再直接使用此样本的 v_c。因此新残差的随机部分仅来自目标 $-v_c$，其方差为 1。在**这个教学示例**中，旧残差方差大于新残差方差。

易错点： 这不意味着所有 Jacobian 和区间长度下新损失的方差都必然更小。实际放大量由 $\Delta J_z u_\theta$ 决定，具体训练还受网络预测误差、权重和时间采样影响。作者在自己的对照实验中观察到旧法损失波动明显更大。

改进思路：用网络近似整体速度。 因为真值 $v(z_t, t)$ 难以计算，iMF 令

$$v_\theta(z_t, t) \approx v(z_t, t), \tag{2.13}$$

然后使用

$$g_\theta^{\mathrm{new}} = \mathrm{JVP}\left(u_\theta; (z_t, r, t); (v_\theta(z_t, t), 0, 1)\right). \tag{2.14}$$

构造复合预测函数

$$V_\theta^{\mathrm{new}} = u_\theta(z_t, r, t) + (t - r)\,\mathrm{sg}(g_\theta^{\mathrm{new}})\,. \tag{2.15}$$

训练目标仍来自已有样本速度：

$$\mathcal{L}_{\text{iMF}} = \mathbb{E}\big\|V_\theta^{\text{new}} - (\epsilon - x)\big\|_2^2. \tag{2.16}$$

现在固定 (z_t, r, t) 后，V_θ^{new} 不再依赖同一样本的 $\epsilon - x$ 标签，具有更标准的回归预测函数形式。

为什么这在回归意义上更自然？ 对于固定输入，令 V_θ^{new} 为确定的预测，仍可使用式(1.8)的平方误差分解：

$$\mathbb{E}\big[\|V_\theta^{\text{new}} - v_{\text{c}}\|^2 \mid z_t, r, t\big] \tag{2.17}$$

$$= \|V_\theta^{\text{new}} - v(z_t, t)\|^2 + \operatorname{tr}\big(\operatorname{Cov}(v_{\text{c}} \mid z_t, t)\big). \tag{2.18}$$

最后的方差项与网络给出的预测数值无关；所以作者将这种形式视为更标准的速度回归。要注意 v_θ 本身可能有估计误差，不能把「随机波动少」误认为「任何阶段都更准确」。

本节要点： iMF 的核心算法变化是把 JVP 的方向从样本速度 v_{c} 换成预测的整体速度 v_θ。v-loss 的代数改写帮助发现问题，但**独立来看不构成改进**。

2.4 两个预测量意味着训练两个大模型吗？

方案 A：同一个网络复用边界条件。 上一章已经证明

$$u(z_t, t, t) = v(z_t, t)$$

（严格地说来自区间长度趋于零的连续延拓）。所以可以定义

$$v_\theta(z_t, t) \triangleq u_\theta(z_t, t, t). \tag{2.19}$$

训练时用**同一组网络参数**分别计算 $u_\theta(z_t, r, t)$ 和 $u_\theta(z_t, t, t)$；它不是训练两套独立网络。不过，同一网络需要额外求值以获得 v_θ，所以训练成本也不一定与原法完全相同。

方案 B：共享主干，加一个辅助 v 预测头。 令共享特征为 $h_\theta(z_t, \cdots)$，平均速度头和瞬时速度头分别读出：

$$u_\theta = \text{Head}_u(h_\theta), \tag{2.20}$$

$$v_\theta = \text{Head}_v(h_\theta). \tag{2.21}$$

这是**一个共享主干、两个输出分支**，而不是必然两个完整的 Transformer。iMF v2 源码附录说明：辅助头与主网络共享大部分层，仅最后 $L = 8$ 层使用独立参数。

因为新分支的一部分层不共享参数，而 g_θ^{new} 在主 Loss 中被 Stop Gradient 截断，辅助头还要直接回归瞬时速度：

$$\mathcal{L}_{\text{aux}} = \mathbb{E}\big\|v_\theta(z_t, t) - (\epsilon - x)\big\|^2. \tag{2.22}$$

实际实现还采用作者所述的自适应损失权重；不能简单断言所有分支仅靠(2.16)就能有效训练。

论文 Algorithm 1 与最终配置为何看起来不同？ **Algorithm 1 展示方案 A**：先计算 $v = u_\theta(z_t, t, t)$，然后作为 JVP 方向；实验还对照**方案 B 辅助头**，最终模型的附录配置列出了 8 层辅助头。两者都是论文提出并验证的实现，并不矛盾。

推理时还需要 v_θ 吗？ 不需要。无论训练中是同一网络复用还是辅助头，推理仍然是

$$x_{\text{pred}} = \epsilon - u_\theta(\epsilon, 0, 1).$$

One-Step 的「一步」指推理期 NFE=1，不约束训练期间可以计算多少次网络输出。

2.5 Stop Gradient 在 iMF 中被删除了吗？

没有。请对照

$$\underbrace{\|u_\theta - \text{sg}(v_c - \Delta g_\theta)\|^2}_{\text{原始 MF 写法}} \quad \longrightarrow \quad \underbrace{\|u_\theta + \Delta\,\text{sg}(g_\theta) - v_c\|^2}_{\text{iMF 的 v-loss 形式}}.$$

改变的是 Stop Gradient 的**位置描述**：旧式写在目标 u_{tgt} 中，新式写在复合预测 V_θ 的 JVP 分支中。保留它可以避免反向传播时继续对 JVP 求参数导数，引入额外高阶求导。

半梯度与完整梯度仍然不同。 若去掉 sg，则 $\partial_\theta g_\theta$ 会参与更新；如果 g_θ 依赖 v_θ，还可能引入经过速度预测分支的额外梯度路径。论文明确说明在其实现中保留 Stop Gradient 更容易优化，**并不认为 Stop Gradient 在所有数学优化目标中都是必要的**。

2.6 Flexible CFG：为什么另有第二项改进

CFG 是什么？ Classifier-Free Guidance（无分类器引导，CFG）通过组合「有条件」与「无条件」的瞬时速度，增强生成对条件 c 的响应。定义

$$v_{\text{cfg}}(z_t, t \mid c) = \omega\, v(z_t, t \mid c) + (1-\omega)\, v(z_t, t). \tag{2.23}$$

其中 ω 是引导尺度：$\omega = 1$ 对应普通条件速度，$\omega > 1$ 通常更强调条件，但并不保证数值越大效果越好。叫作「无分类器」是因为不必训练一套额外分类器来提供引导梯度；**它不是没有条件**。

为什么标准两次网络调用的 CFG 不适合直接照搬？ 标准 CFG 在每一步都计算有条件与无条件预测，通常意味着两次网络前向；这与 MeanFlow 希望 1-NFE 的设计目标冲突。更深层的是：**对瞬时速度的引导会改变生成轨迹，不能无条件认为「两个平均速度做同样的线性组合」就是引导后新轨迹的平均速度**。

原始 MeanFlow 的限制。 原始 MF 已经尝试**在训练时学习 CFG 速度场对应的平均速度**，因此能够在 1-NFE 推理阶段只输出一次；但它的引导强度 ω 在训练阶段固定，使用上不够灵活。

iMF 的做法：把引导尺度变成条件变量。 训练时随机抽取 ω，让网络学习随 ω 变化的平均速度：

$$u_\theta\big(z_t, r, t \mid c, \omega\big). \tag{2.24}$$

推理时只选择希望使用的 ω，仍然调用一次网络：

$$x_{\text{pred}} = \epsilon - u_\theta(\epsilon, 0, 1 \mid c, \omega). \tag{2.25}$$

更进一步，iMF 把引导区间也纳入条件：

$$\Omega = \{\omega, t_{\min}, t_{\max}\}. \tag{2.26}$$

训练时对 ω 和区间端点采样，让模型适应不同引导策略；在区间外设 $\omega = 1$ 表示关闭额外引导。论文源码中采样 $\omega \in [1, 8]$，并对较小引导尺度赋予更高采样权重。

iMF 使用的 CFG 速度监督如何构造？ 论文附录根据原始 MF 的引导推导，使用一种与引导尺度相关的有效速度目标（此处沿用论文的 ω 约定）：

$$v_{\text{tgt}} = v_{\text{c}} + \left(1 - \frac{1}{\omega}\right)\left(u_\theta^{\text{cfg}}(z_t \mid t, t, c) - u_\theta^{\text{cfg}}(z_t \mid t, t, \varnothing)\right). \tag{2.27}$$

若 $\omega = 1$，修正项消失，退化为无额外 CFG 的速度目标 v_{c}。这说明**真正的单步 CFG 训练，不是仅把 ω 当作一个输入数字就全部完成**；训练目标还要对应引导后的整体速度场。式(2.27)来自 iMF 原文附录，而不是由普通的(2.23)直接逐样本替换而得。

原文对应：iMF 源码 `sections/method.tex` “Flexible Guidance” 和 `sections/appendix.tex` “CFG conditioning”。

2.7 In-context Conditioning：模型怎样接收这么多条件

当网络需要同时理解 (r, t)、类别 c 和 Ω 时，条件很多。传统 DiT 常使用 adaLN-zero 把条件编码注入网络。iMF 改为把不同条件变成**多个条件 Token**，与图像潜变量 Token 拼接，再交给 Transformer 一起处理。

论文源码的示例配置是：类别 8 个 Token，时间和引导相关条件分别使用 4 个 Token。这些条件经位置编码、MLP 与类型嵌入处理。作者移除较重的 adaLN-zero 模块，配合其初始化方案，在 B-size 对照中将参数量从 133 M 降到 89 M。

易错点： 网络参数减少不等于实际推理时延按同一比例减少；Condition Token 也会增加 Transformer 序列长度与计算。iMF 的最终 FID 提升是多个因素叠加的结果，不应全部归因于 JVP 方向修正。

2.8 论文怎样验证改进？

速度方向的消融。 论文 Table 1(a) 在 ImageNet 256×256、1-NFE 条件下，比较了原法与两种 v_θ 实现：

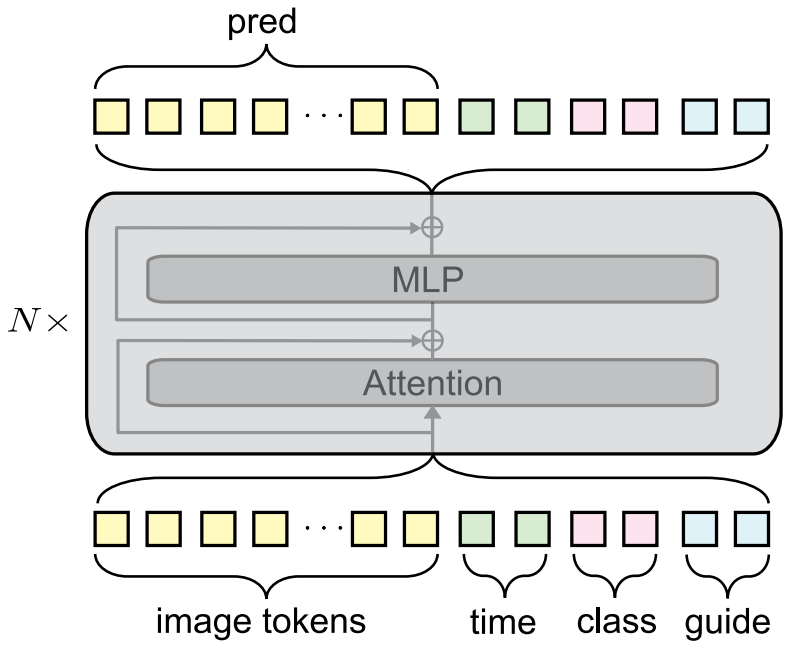


图 2.2: iMF 的 In-context Conditioning 结构原图，直接来自 arXiv:2512.02012v2 源码 `figs/arch.pdf`，未重绘。

方法	无 CFG FID↓	有 CFG FID↓
原始 MeanFlow	32.69	6.17
复用边界 $u_\theta(z_t, t, t)$	**29.42**	5.97
辅助 v-head	30.76	**5.68**

这说明**辅助头并非处处胜出**：在有 CFG 的对照中辅助头较好，但无 CFG 时边界方案较好。论文 Figure（Training losses）还发现，在其测试条件下旧方法的非零区间 Loss 波动更大；这支持动机，但不等于有普适收敛定理。

完整系统的结果。 对于最终 iMF-XL/2，论文报告 ImageNet 256×256 的 1-NFE FID 为 1.72，对照原始 MF-XL/2 的 3.43。**这是整套系统的结果，不是只有改动 JVP 一个变量的消融结果**。本书不将图像生成 FID 直接解释为机器人动作策略性能。

原文对应：iMF 原文 Table 1、完整模型对照表及 Experiments 部分；源码对应实验与表格文件。

2.9 iMF 训练伪代码与最后的疑问

边界条件方案：与原文 Algorithm 1 对齐。

```
# x: real-data batch
# fn(z, r, t): shared network predicting u
# conceptual pseudocode; tensor broadcasting omitted
r, t = sample_interval()
eps = randn_like(x)
z_t = (1 - t) * x + t * eps
v_sample = eps - x

v_pred = fn(z_t, t, t)   # predict marginal v
u_pred, du_dt = jvp(
    fn, (z_t, r, t), (v_pred, 0, 1)
)
V_pred = u_pred + (t - r) * stopgrad(du_dt)
loss = mse(V_pred, v_sample)
update_model(loss)
```

辅助头方案用 v-head 的 v_θ 替代 `fn(z_t,t,t)`，并加上辅助 Flow Matching loss；若使用 CFG，还需要相应的引导目标与条件输入。实际代码还应处理 batch 形状、自适应权重及 Stop Gradient 的具体作用路径。

问：iMF 训练的是不是 v，而不再训练 u 了？ 不是。它**仍然训练** u_θ，最终单步生成也只需要 u_θ。v-loss 指的是监督残差在**瞬时速度空间**中度量；不是说主生成网络换成了普通 Flow Matching。

问：预测速度 v_θ 在训练初期也不准，为何敢用？ 这是合理的担忧。v_θ 可能有偏差，所以 iMF 不是获得了真实整体速度；它是用**对当前输入较一致、通常随机性更低的预测方向**替换每条样本的随机方向，并通过边界训练或辅助 FM loss 促进 v_θ 学习。收益需要实验验证。

问：如果去掉 Stop Gradient 是否仍是合法的 Loss？ 可以定义完整梯度形式，但它与作者采用的半梯度更新不同，额外高阶导数还会增加优化复杂度。论文的工程结论是在其设置中保留 Stop Gradient 更合适。

第 3 章

Flow Matching、MeanFlow 和 iMF 对比

3.1 核心数学对象与训练机制总表

下面所有公式统一使用本书的时间方向：$t=0$ 是数据端，$t=1$ 是噪声端；三者使用同一个线性插值 $z_t=(1-t)x+t\epsilon$，样本速度都可取 $v_{\mathrm{c}}=\epsilon-x$。

对比项	普通 Flow Matching	MeanFlow（MF）	Improved Mean Flows（iMF）
核心问题	学瞬时速度场	学区间平均速度场	改进平均速度网络的训练与引导
主网络预测	$v_\theta(z_t, t)$	$u_\theta(z_t, r, t)$	$u_\theta(z_t, r, t \mid c, \Omega)$
真正的理论量	$v = \mathbb{E}[v_{\mathrm{c}} \mid z_t, t]$	$u = (t-r)^{-1}\int_r^t v\ \mathrm{d}\tau$	与 MF 相同的 u
已知样本信号	$v_{\mathrm{c}} = \epsilon - x$	$v_{\mathrm{c}} = \epsilon - x$	$v_{\mathrm{c}} = \epsilon - x$（CFG 时用引导目标）
数学关键	条件期望回归	$u = v - (t-r)\,\mathrm{d}u/\,\mathrm{d}t$	$v = u + (t-r)\,\mathrm{d}u/\,\mathrm{d}t$
JVP 需求	标准 FM Loss 不需要	要；方向为 $(v_{\mathrm{c}}, 0, 1)$	要；方向为 $(v_\theta, 0, 1)$
回归量	v_θ 对 v_{c}	u_θ 对 u_{tgt}	V_θ 对 v_{c}
Stop Gradient	基本 FM Loss 不需要	训练目标分支中使用	复合预测的 JVP 分支中使用
瞬时速度预测	主网络直接给出	不另设 v_θ	复用 $u_\theta(z_t, t, t)$ 或辅助 v-head
训练时额外计算	普通前向与反向	JVP 及一次参数更新	JVP 与 v_θ 估计；辅助头方案另有 FM loss
生成方法	数值积分 ODE	直接平均速度更新	与 MF 相同的平均速度更新
1-NFE 能力	可以强行用 Euler 一步，但可能误差较大	专门训练以支持 1-NFE	同样专门支持 1-NFE
CFG 策略	通常逐步做有条件/无条件组合	单步 CFG 可在训练期固定引导强度	ω 和时间区间作为条件、推理时可调
主要局限	多步推理昂贵	JVP 含样本速度，Loss 可能高方差	仍依赖 v_θ 估计；训练与结构更复杂

3.2 最关键的三组损失，逐式对照

普通 Flow Matching：直接拟合采样速度。

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}\|v_\theta(z_t, t) - (\epsilon - x)\|^2. \tag{3.1}$$

它预测的是瞬时速度，因此通常需要多次预测才能积分得到整个生成位移。

MeanFlow：预测平均速度，目标中含 JVP。

$$g_\theta^{\text{old}} = \text{JVP}\left(u_\theta; (z_t, r, t); (\epsilon - x, 0, 1)\right), \tag{3.2}$$

$$\mathcal{L}_{\text{MF}} = \mathbb{E}\left\| u_\theta - \text{sg}\Big((\epsilon - x) - (t - r) g_\theta^{\text{old}}\Big)\right\|^2. \tag{3.3}$$

其标签 u_{tgt} 依赖当前网络的 JVP，训练属于自举式半梯度优化。

iMF：预测平均速度，但在瞬时速度空间回归。

$$g_\theta^{\text{new}} = \text{JVP}\left(u_\theta; (z_t, r, t); (v_\theta, 0, 1)\right), \tag{3.4}$$

$$V_\theta = u_\theta + (t - r)\,\text{sg}(g_\theta^{\text{new}})\,, \tag{3.5}$$

$$\mathcal{L}_{\text{iMF}} = \mathbb{E}\|V_\theta - (\epsilon - x)\|^2. \tag{3.6}$$

这里的 V_θ 由 u_θ 重参数化而来；**不是普通 FM 的 v_θ，也不是推理时需要另调用的第二个完整生成模型。**

3.3 训练与推理各需要什么：快速检查表

步骤	FM	MF	iMF
采样 x, ϵ, t	✓	✓	✓
额外采样 r	不需要	需要	需要
计算 $v_{\text{c}} = \epsilon - x$	✓	✓	✓
训练时求 JVP	不需要	需要	需要
为 JVP 预测 v_θ	不需要	不需要	需要
使用 Stop Gradient	基本 Loss 不需要	需要	需要
推理时需要真实 x	不需要	不需要	不需要
推理时需要 JVP	不需要	不需要	不需要
标准单步推理操作	Euler 粗近似	$\epsilon - u_\theta$	$\epsilon - u_\theta$

3.4 你应该能回答的七个自检问题

1. 为什么 $z_t = (1-t)x + t\epsilon$ 的采样路径速度是常数，边缘速度场 $v(z_t, t)$ 却可以变化？
   提示：同一个 z_t 可以由不同的 x, ϵ 产生。
2. 为什么 $\mathrm{d}/\mathrm{d}t \int_r^t v(z_\tau, \tau)\,\mathrm{d}\tau = v(z_t, t)$？
   提示：把新增的短区间积分除以 Δt。
3. MeanFlow Identity 的 $\mathrm{d}u/\mathrm{d}t$ 为什么不能只写 $\partial_t u$？
   提示：z_t 也随时间变化。
4. JVP $(u_\theta; (z_t, r, t); (v_c, 0, 1))$ 的三个方向分量分别表示什么？
   提示：分别是 $\mathrm{d}z_t/\mathrm{d}t$、$\mathrm{d}r/\mathrm{d}t$、$\mathrm{d}t/\mathrm{d}t$。
5. Stop Gradient 是不是让模型参数完全不更新？
   提示：只有目标/*JVP* 分支停止，预测分支仍更新。
6. 仅将 MF 的 u-loss 移项写为 v-loss，会不会改变梯度？
   提示：保持 *JVP* 方向与 *Stop Gradient* 规则时，两者等价。
7. iMF 为什么还要预测 v_θ，它推理阶段是否还要用？
   提示：它在训练时提供更接近整体速度场的 *JVP* 方向，推理只需要 u_θ。

3.5 结论：真正的技术演进

Flow Matching： 学习整体瞬时速度场，用多次网络调用数值积分，得到从噪声到数据的轨迹。

MeanFlow： 把目标从瞬时速度改为**区间平均速度**。通过对平均速度定义求导得到 MeanFlow Identity，再用 JVP 与 Stop Gradient 构造自举目标，使推理阶段可以一次跨越整个区间。

Improved Mean Flows： 保留上述数学框架。**第**一，将原方法等价重述为 v-loss，发现 JVP 接收样本速度作为额外输入的问题；**第**二，用预测的整体速度 v_θ 取代随机样本速度作为 JVP 方向；**第**三，通过可调 CFG 条件和多 Token 条件输入改善可控性与整体生成效果。

复习时最应记住的关系。

$$
\begin{aligned}
&\text{FM：} && v_{\mathrm{c}} \longrightarrow v_\theta && \text{（瞬时速度），}\\
&\text{MF：} && v_{\mathrm{c}},\, \mathrm{JVP}(u_\theta; v_{\mathrm{c}}) \longrightarrow u_\theta && \text{（平均速度），}\\
&\text{iMF：} && v_{\mathrm{c}},\, \mathrm{JVP}(u_\theta; v_\theta) \longrightarrow u_\theta && \text{（更标准的速度回归）.}
\end{aligned}
$$

上式是逻辑缩写，精确损失以本章 §3.2为准。无论 MF 还是 iMF，最终一步生成都由(1.16)完成。

论文与源码出处

1. Geng et al. (2025). *Mean Flows for One-step Generative Modeling*. arXiv:2505.13447v1. `https://arxiv.org/abs/2505.13447`。正文主要对应 Sec. 3、4.1、4.2、4.3 与算法 1–2。
2. Geng et al. (2026). *Improved Mean Flows: On the Challenges of Fastforward Generative Models*. arXiv:2512.02012v2. `https://arxiv.org/abs/2512.02012`。数学式、CFG 推导、消融表和所收录图片直接核对上传的 arXiv LaTeX 源码。
3. 参考排版工程：用户提供的 `flow_matching_latex.zip`。本书沿用其 `ctexbook`、`style.tex`、`math_macros.tex` 结构；内容为重新撰写的学习笔记。